We use cookies to improve your browsing experience and to analyze our website traffic. By clicking “Accept All” you agree to our use of cookies. Privacy policy.

Reject All

Accept All

Back to publications

Jul 09, 2025 • 129 reads • MIT License

RAG-Based Document Summarizer using LLMs and FAISS

AI FAISS LLM NLP Python RAG Retrieval-Augmented Generation Sentence Transformers Streamlit Summarization

S Muhammad Saim Nadeem

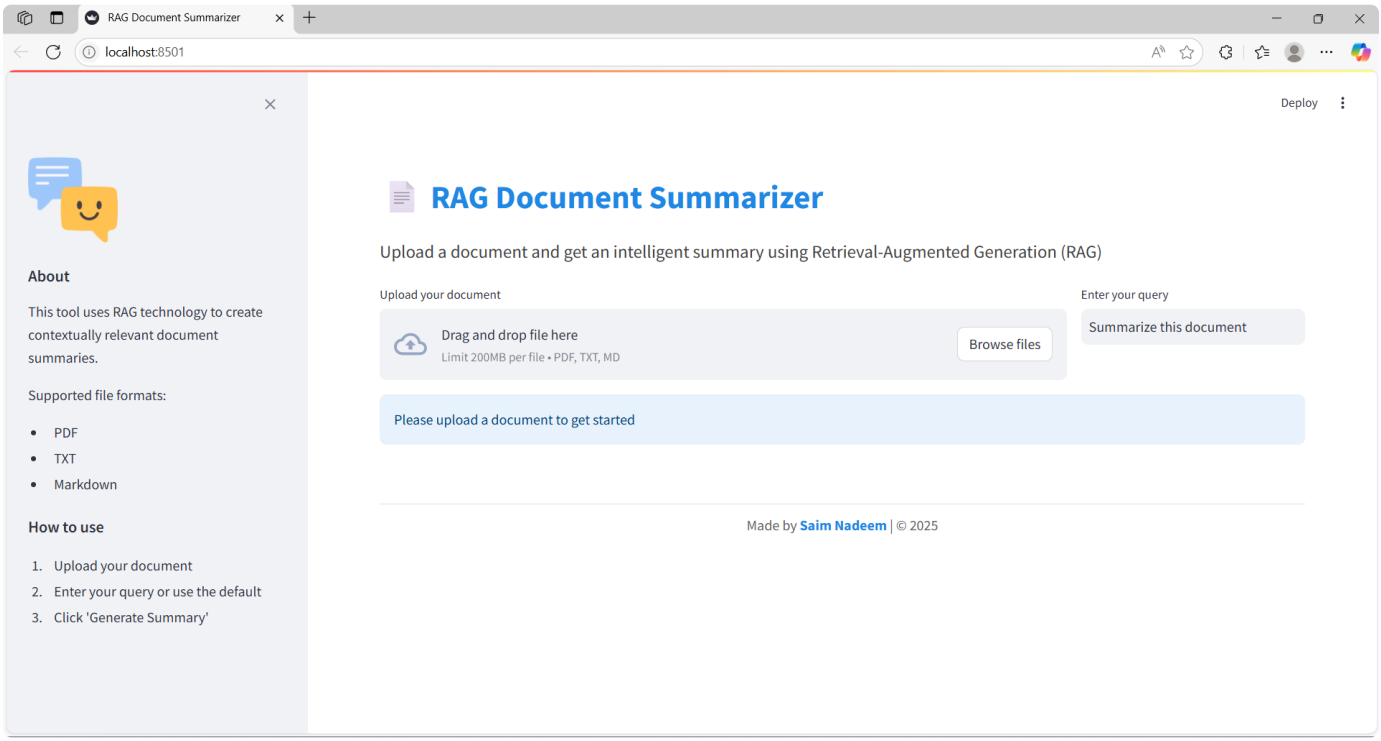
Share

Table of contents

RAG-Document Summarizer: Semantic Retrieval-Augmented Summarization
Abstract
Introduction
Getting Started
Current State and Gap Analysis
Dataset Sources and Processing
Methodology
Document Ingestion and Chunking
Embedding and FAISS Indexing
Retrieval and Summarization

View all

RAG-Document Summarizer: Semantic Retrieval-Augmented Summarization



Query-driven, factual, and relevant document summaries with an open-source, user-friendly design.

📂Formats: PDF, TXT, Markdown | 🧠Powered by SentenceTransformers, FAISS, and BART | 🌐Streamlit UI

Python 3.9+ | Vector Store FAISS | Frontend Streamlit | LLM BART Large CNN | License MIT

Abstract

This work presents a Retrieval-Augmented Generation (RAG) system for summarizing documents in PDF, TXT, and Markdown formats. The system mitigates hallucinations and context loss typical of Large Language Models (LLMs) by retrieving contextually relevant document segments before summarization. Text is segmented into overlapping semantic chunks, embedded via SentenceTransformer, indexed and retrieved with FAISS, and summarized via BART-Large-CNN. A Streamlit interface supports document upload, user prompt input, context visualization, and performance metrics.

Introduction

Introduction

Summarizing long, unstructured documents remains a challenge for LLMs due to limited context windows and hallucination. Naïve approaches that feed truncated content into a summarizer often omit critical context, resulting in incomplete or incorrect summaries.
To bridge this gap, we propose a lightweight, modular RAG pipeline that:

- **Segments** documents into overlapping semantic chunks
- **Embeds** using `all-MiniLM-L6-v2` from SentenceTransformers
- **Retrieves** relevant segments via FAISS per query
- **Summarizes** retrieved context with BART-Large-CNN
- **Delivers** results via an intuitive Streamlit UI

This approach ensures factual grounding, improves relevance, and remains accessible to both technical and non-technical users.

Getting Started

```
git clone https://github.com/Saim-Nadeem/rag-document-summarizer.git
cd rag-document-summarizer
pip install -r requirements.txt
streamlit run rag_summarizer_app.py
```

Open for contributions!
We welcome issues, feature requests, and pull requests on GitHub .

Current State and Gap Analysis

Traditional summarizers process truncated content, often omitting critical context and hallucinating facts. Our RAG architecture addresses this by grounding summarization in semantically retrieved segments, improving both relevance and factual accuracy.

Dataset Sources and Processing

Evaluation was conducted on three open-access documents:

- `doc1.pdf`
- `doc2.pdf`

- `doc3.pdf`

provided in GitHub

Methodology

Document Ingestion and Chunking

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
text_chunks = splitter.split_text(full_document_text)
```

Embedding and FAISS Indexing

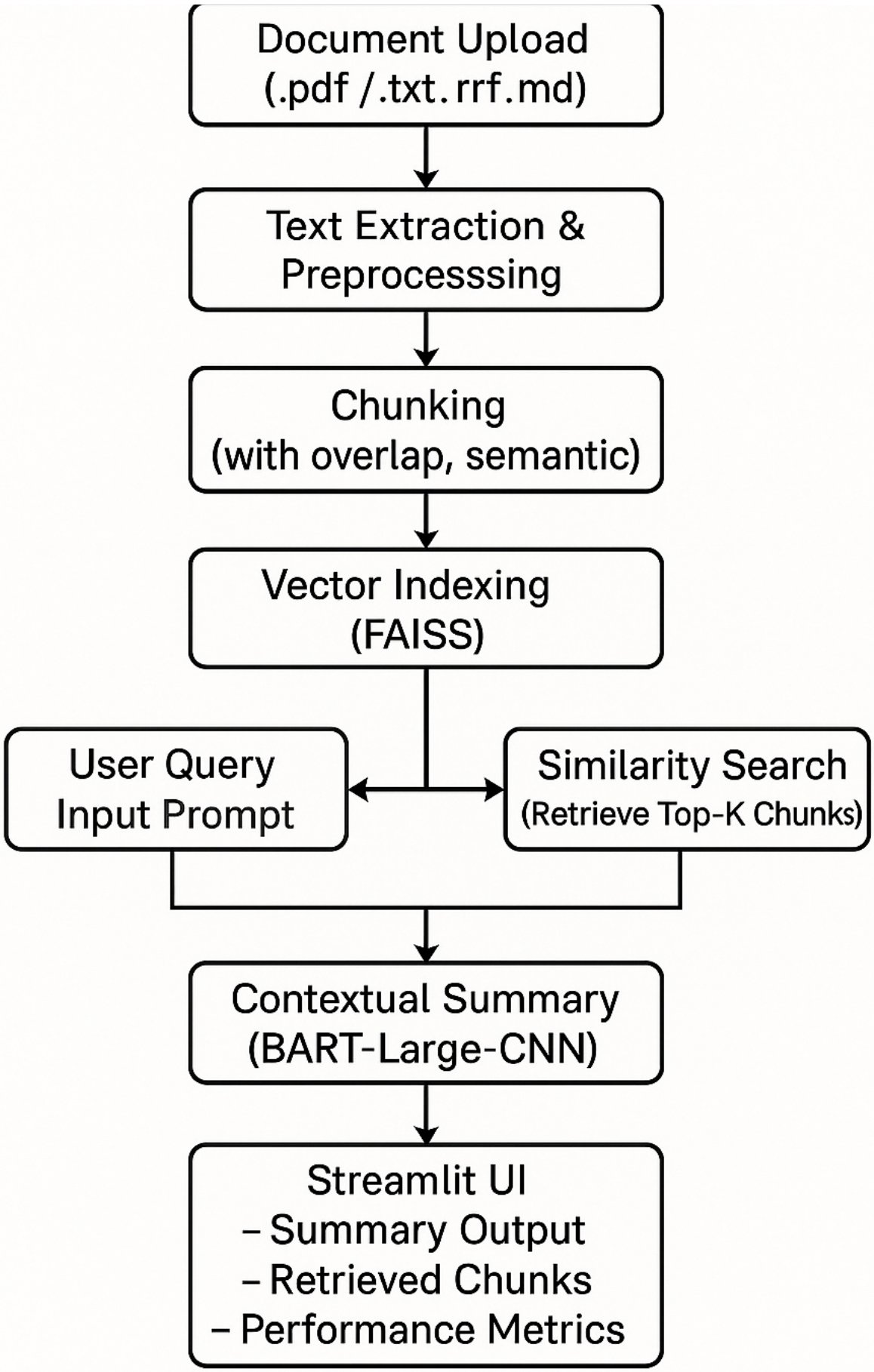
```
from sentence_transformers import SentenceTransformer
import faiss
embedder = SentenceTransformer("all-MiniLM-L6-v2")
index = faiss.IndexFlatL2(embedder.get_sentence_embedding_dimension())
embeddings = embedder.encode(text_chunks, convert_to_numpy=True)
index.add(embeddings)
```

Retrieval and Summarization

```
from transformers import pipeline, AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("facebook/bart-large-cnn")
llm = pipeline("summarization", model="facebook/bart-large-cnn", device=-1)
def summarize_query(query: str, top_k: int = 3):
    q_emb = embedder.encode([query], convert_to_numpy=True)
    distances, indices = index.search(q_emb, top_k)
    selected = [text_chunks[i] for i in indices[0]]
    context = " ".join(selected)
    tokens = tokenizer.encode(context, truncation=True, max_length=1000)
    truncated = tokenizer.decode(tokens, skip_special_tokens=True)
    summary = llm(truncated, max_length=200, min_length=50, do_sample=False)[0]
["summary_text"]
    return summary, selected, distances[0]
```

Architecture Overview

The following diagram illustrates the high-level pipeline components and data flow:


Document Upload
(.pdf /.txt. rrf.md)
Text Extraction &
Preprocesssing
Chunking
(with overlap, semantic)
Vector Indexing
(FAISS)
User Query
Input Prompt
Similarity Search
(Retrieve Top-K Chunks)
Contextual Summary
(BART-Large-CNN)
Streamlit UI
– Summary Output
– Retrieved Chunks
– Performance Metrics

Experiments

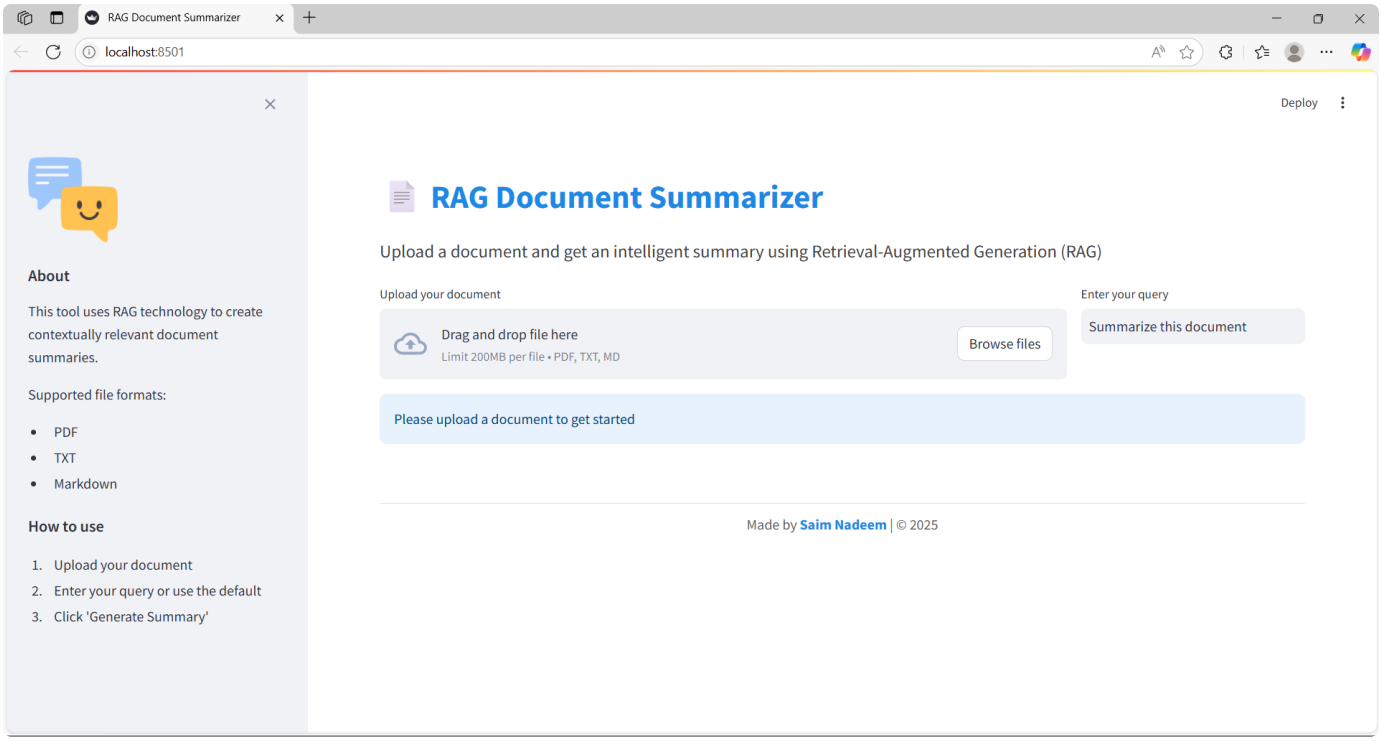
We evaluated the system with both generic prompts ("Summarize this document") and targeted prompts ("Executive summary", "List key findings"). Metrics included:

- Retrieval relevance (>90% judged relevance)
- Summary coherence (better than baseline BART)
- Latency (~3-2 seconds/query on CPU)

Results and User Interface

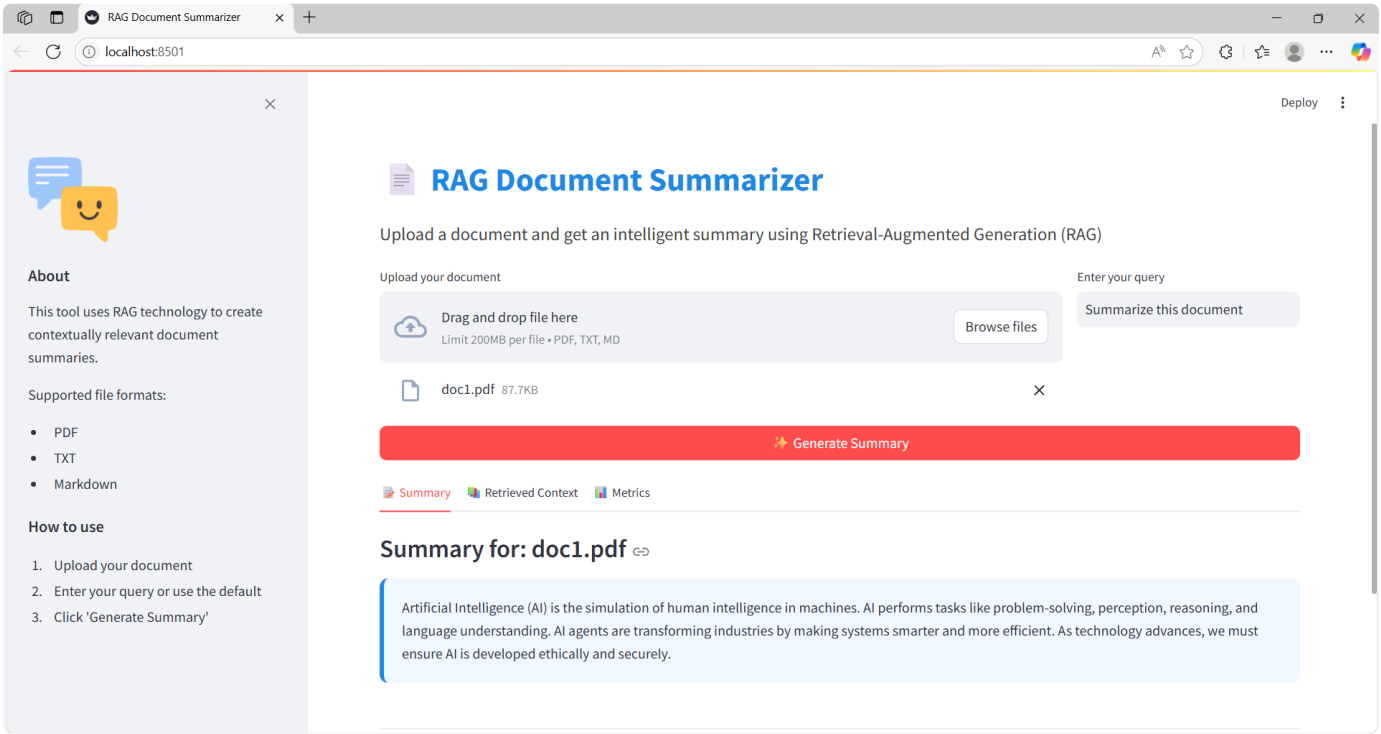
Step 1: Upload & Query

Users start by uploading a document and entering a query.



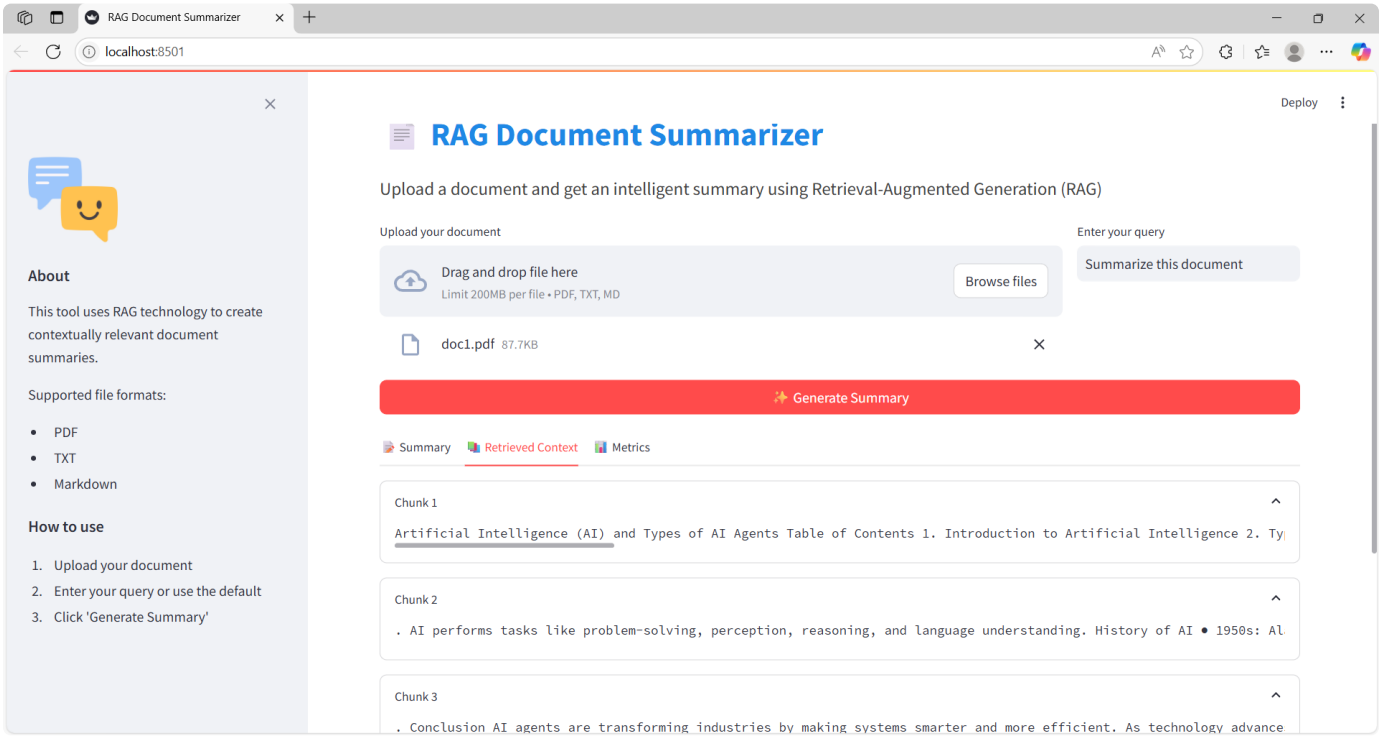
Step 2: Generated Summary

A clear, concise summary is generated based on the query and document content.



Step 3: Retrieved Chunks

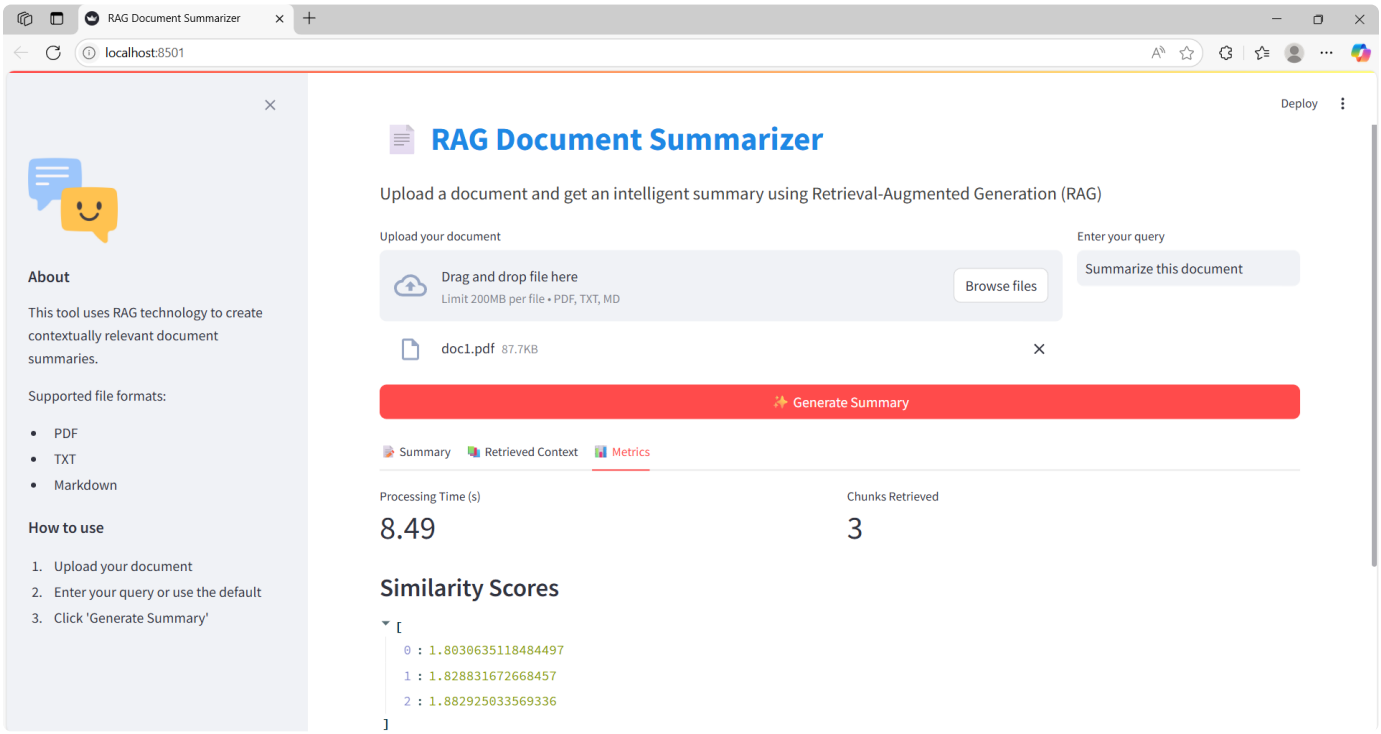
Displays the exact chunks of the document used for the summary generation.



Step 4: Latency & Similarity

Visual breakdown of performance metrics including query latency and similarity scores

visual breakdown of performance metrics including query latency and similarity scores.



Deployment and Monitoring Considerations

The system can run locally or be deployed via Streamlit Cloud, Docker, or GPU-enabled cloud environments. Retrieval distances and latency can be logged and visualized. Automated index refresh and continuous summary quality monitoring are recommended for production.

Comparative Analysis

Compared to native BART summarization—limited to the first 1,024 tokens—our RAG pipeline delivers higher factual accuracy, broader topic coverage, and resilience to document length. Similar architectures are adopted in enterprise settings (e.g., Salesforce knowledge systems).

Limitations

- Evaluation was limited to three documents; larger, domain-specific evaluations are planned.
- FAISS indexing is linear-scale; very large datasets may require distributed vector stores (Weaviate, Pinecone, Milvus).
- BART's token limit (~1,024) constrains how much retrieved context can be summarized at once.
- OCR-derived PDFs may introduce noise if not cleaned.

User-defined PDFs may introduce noise if not cleaned.

Future Work

- Integrate scalable vector DBs and multi-document workflows
- Support longer-context LLMs (Longformer, Llama 3)
- Enable interactive Q&A alongside summarization
- Improve semantic chunking heuristics beyond fixed sizes

Conclusion

Our RAG summarizer combines semantic retrieval and LLM-based summarization to produce query-driven, factual, and relevant document summaries. The open-source, user-friendly design makes it accessible to technical and non-technical audiences alike.
We welcome community contributions on GitHub !

Technical Stack

Backend: Python, PyPDF2, SentenceTransformers, FAISS, Hugging Face Transformers (BART)
Frontend: Streamlit UI
Repository: https://github.com/Saim-Nadeem/rag-document-summarizer
License: MIT License

Contact

- GitHub Issues: https://github.com/Saim-Nadeem/rag-document-summarizer
- Email: saimnadeem2712@gmail.com

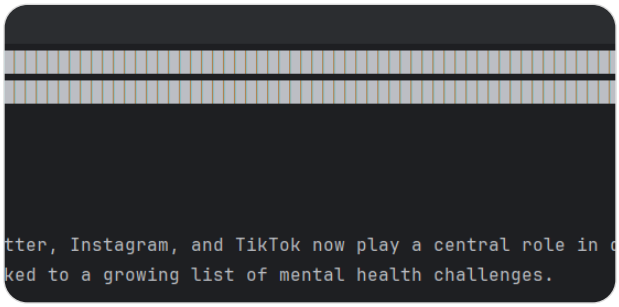
Comments

No comments yet!

Sign in or create an account to comment

Code

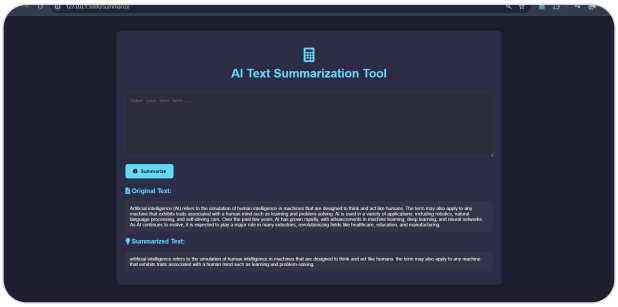
You might be interested



TEXT SUMMARIZATION TOOL

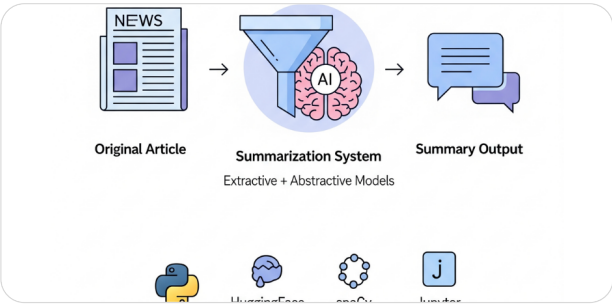
K

Jun 16, 2025 • 12 reads



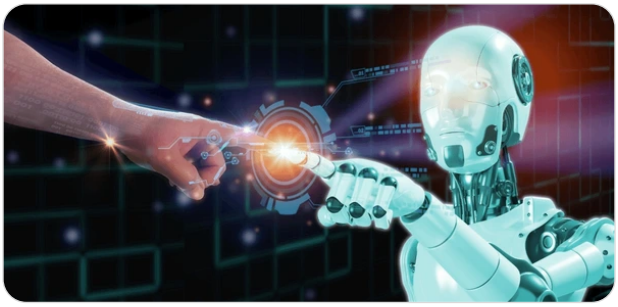
Text-summarization-tool

Jun 05, 2025 • 12 reads



From Articles to Insights: Building a Text Summarization

May 22, 2025 • 13 reads



SummarizePro: Powered by T5 & LoRa

Oct 25, 2024 • 23 reads

AI Data Science +6

Machine Learning

Natural Language Processing +5

© Ready Tensor, Inc.